

플레이어 이동 구현 방법 비교

영웅호걸 2차 코드리뷰
- 윤지원 -

지난 코드 리뷰 다시보기

Unity Canvas Rebuild 문제

Unity의 UI 시스템은 **Canvas 단위로 갱신**이 일어나고 런타임에 UI요소가 변경되면 Unity는 Canvas를 **'Rebuild'**합니다.

Rebuild는 Canvas 전체의 배치, 정렬, 드로우콜을 다시 계산하는 작업을 수행합니다.

매 프레임마다 시간을 표시하는 Text UI가 변경되면 해당 Text가 속하는 Canvas의 모든 요소가 Rebuild 대상입니다 ..!!!! (SetActive도 포함)

따라서, Canvas를 기능 단위로 분리하거나 자주 바뀌는 UI는 따로 분리해, 전체 갱신 부담을 줄여야 합니다.

그럼 무조건 Canvas를 나누는 것이 좋은가?! => NONONO

UI Batching은 같은 캔버스 내에서 Texture, Material, Shader 상태가 같고 Hierarchy 상으로 연속된 위치에 있는 UI를 한 묶음으로 렌더링하기 때문에 Canvas가 너무 많이 나뉘어져 있으면 최적화 면에서 좋지 않습니다.

그럼 언제 써야하는가!?!?

“**UI가 자주 변경되는가 ?**”, “**같은 Canvas에 정적 UI가 많이 있는가 ?**” => Canvas 분리 고려

Unity Canvas Rebuild 문제

과거 UI 구조



변경된 UI 구조

PF_UIRoot

HUDLayer

- PlayerStatus(Canvas)
 - HPBar
 - StaminaBar
 - Timer

(변경이 잦은 UI를 모음)

- Inventory Icon
- 그 외 기타 등등

중첩 Canvas 없이 각 layer
하위에 UI 오브젝트를 넣을
계획이었으나,

HUD Layer에서 자주 변경되는 HP, Timer 등을
별도의 Canvas에 넣는 것으로 설계 변경

UI Layer 렌더 순서 고정

현재 구조는 **UIManager**가 **UI**를 배치하도록 하여 **Instantiate**를 한 곳에 모으고 **UI**생성, **Layer**배치, 렌더 순서, **Open/Close** 들을 중앙에서 통제하게끔 설계했다.

이 구조로 **UI**는 자신이 어디에 배치될지를 알 필요없이 어떤 종류의 **UI**인지만 알 수 있어서 의존성이 줄어든다.

Unity UI나 **Canvas** 자체에서도 렌더 순서를 정할 수 있지만, 위 설계를 지키기 위해 각 **UI**들이 본인이 어떤 **Layer**에 속하는 **UI**인지를 명시하고, **UIManager**는 그 타입에 따라 **Layer**에 배치합니다. 각 **Layer** 안에서는 **SeblingOrder**에 따라 순서가 결정됩니다.

Addressables를 왜 사용하는가?

Unity에서 런타임에 에셋을 불러오는 방법에는 3가지가 있다!

Resources

- 가장 단순한 방법
- **Resources** 폴더에 에셋을 넣고 경로만 지정하면 바로 사용 가능
- **Resources** 폴더에 있는 에셋을 수정하거나, 변경이 발생하면 전체를 다시 배포하거나 업데이트가 필요함
=> 대규모 리소스 관리가 힘들다

AssetBundle

- **Resources**의 문제를 해결
- 에셋을 여러 파일로 묶어서 필요한 것만 로드하거나 다운로드 할 수 있다.
=> 패키징/다운로드 가능
- **Bundle** 위치, **Dependency**, 로딩 순서, 해제 시점 등을 직접 관리해야 한다.

Addressables

- **AssetBundle**의 기능에서 관리 과정을 추상화한 시스템
- 에셋의 **Address**만 알고 있으면 따로 관리하지 않아도 사용 가능
- **AssetBundle**을 안쓰는 것이 **아니다!** 관리 기능만 추상화한 것.

Addressables를 왜 사용하는가?

Resources

```
</> C#  
  
var prefab =  
    Resources.Load<GameObject>("UI/Inventory");  
  
Instantiate(prefab);
```

- Resources 폴더에 넣고
경로만 입력하면 끝!

AssetBundle

```
</> C#  
  
LoadManifest();  
  
LoadDependencies("ui_inventory");  
  
AssetBundle bundle =  
    LoadBundle("ui_inventory");  
  
GameObject prefab =  
    bundle.LoadAsset<GameObject>("Inventory");  
  
Instantiate(prefab);  
  
AddReference(bundle);
```

- 내가 필요한 에셋이 어느 Bundle에 있는지에 따라 여러 개를 로드해야 할 수 있다. (Bundle 의존성)
- 에셋 로드와 해제도 개발자가...!!!! 포인터에서 하던 Reference Counting을 여기서도 하게될 수 있다...
- 비동기 로딩 시에는 번들관리가 더 까다롭다.!!

Addressables

```
</> C#  
  
var handle =  
    Addressables.LoadAssetAsync<GameObject>(  
        "UI/Inventory"  
    );  
  
GameObject prefab =  
    await handle.Task;  
  
Instantiate(prefab);
```

- 내가 필요한 에셋이 어느 Bundle에 속했는지, 의존성이 몇 개인지, 어떤 Bundle을 먼저 Load 해야하는지 등을 알 필요 없이 주소만 알고있으면 바로 로드 가능
- 다만, 자신이 획득한 Handle이나 인스턴스를 적절하게 Release해야 하는 책임은 남는다.

Addressables를 왜 사용하는가?

패키징이나 리소스 관리를 하기 위해 **Resources** 방식 대신 **AssetBundle**을 사용

그러나, **AssetBundle**은 직접 관리할 것이 많으므로 이러한 관리 과정을 추상화한 **Addressables**도 함께 사용!!

AssetBundle 직접 사용 시

- ✓ **Bundle** 위치 관리
- ✓ **Manifest** 관리
- ✓ **Dependency** 관리
- ✓ **Load** 순서 관리
- ✓ 비동기 로딩 관리
- ✓ **Bundle Cache**
- ✓ **Reference Count**
- ✓ **Unload** 시점 관리



Addressables가 상당 부분 담당

플레이어 이동 구현 방법 비교

플레이어 이동 구현 방법 비교

- Character Controller 컴포넌트
- Rigidbody 물리 이동
- Translate 좌표 위치 직접 이동

Translate

- 좌표를 직접 옮기는 방식
- 물리 충돌이 자동으로 처리되지 않음
- Rigidbody와 충돌 시 문제 발생 가능

Rigidbody : 물리 기반

- 물리 기반 게임에서 사용하기 좋음
- 중력, 항력, 공기저항, 마찰, 반동 등의 물리 시뮬레이션을 제공



Human Fall Flat



Fall Guys

Character Controller : Kinematic 방식

Kinematic(동역학) : 힘을 고려하지 않고 물체의 움직임 자체만 다루는 운동학. 물체의 질량을 따지지 않고 오직 ‘어떻게 움직이는가’에 집중

- 계단 및 경사로를 처리할 수 있으며, 벽에 끼지 않는다.
- 즉각적이고 세밀한 조작을 만들기 쉽다.
- 관절, 마찰, 표면, 힘 등 물리 상호작용은 제공되지 않음 => 중력같은 물리 연산을 직접 계산해야 함
- 다른 Rigidbody컴포넌트와 상호작용이 제한됨